API Performance Monitoring

Logs → ETL → MySQL → Analytics → Alerts → Power BI Dashboard

Karthick Raja

Ingestion and Validation

- Logs are **too many and messy**
  → Hard to understand directly
- Monitoring is **manual and slow**
  → We react *after* issues happen
- Bad data is **hidden or ignored**
  → Causes wrong analysis

```
ip,endpoint,status,timestamp,execution_time
192.168.176.140,/profile,500,2026-03-23 16:02:59.455967,403
```

Proposed Solution

Build a **complete monitoring pipeline**
→ From logs → database → dashboard

Separate: ✅ Clean data | ❌ Rejected (bad) data

- Use SQL to calculate:
  - **KPIs (success rate, latency)**
  - **Risk metrics (slow queries, errors)**
- Show everything in **Power BI dashboard**

Updated End-to-End Pipeline (Final SQL Flow)

- Raw events are generated continuously and written to data/raw/log.csv.
- Business and technical validation separates clean records from rejected records.
- **system_logs, rejected_logs,** and **etl_metrics** maintain analytic and governance state.
- SQL scripts compute **KPI, trend, tail-latency,** and **risk metrics** for dashboards.

Core Tables

1. **system_logs:** request-level fact table.
2. **rejected_logs:** invalid rows with reason.
3. **etl_metrics:** run totals and freshness.
4. **system_logs_archive:** long-term retention.

Data Quality and Speed

- CHECK constraints protect data quality.
- FK links rejected rows to ETL run.
- **Indexes speed time and endpoint queries.**
- Orphan check confirms audit integrity.

Basic, KPI, Advanced, Diagnostics

- **Basic:**
  i. traffic,
  ii. reliability,
  iii. latency.
- **Trend:** spikes by minute and hour.
- **Endpoint:**
  i. slow
  ii. failure-prone APIs.
- **Advanced:**
  i. latency buckets,
  ii. risk scoring,
  iii. root-cause guidance.

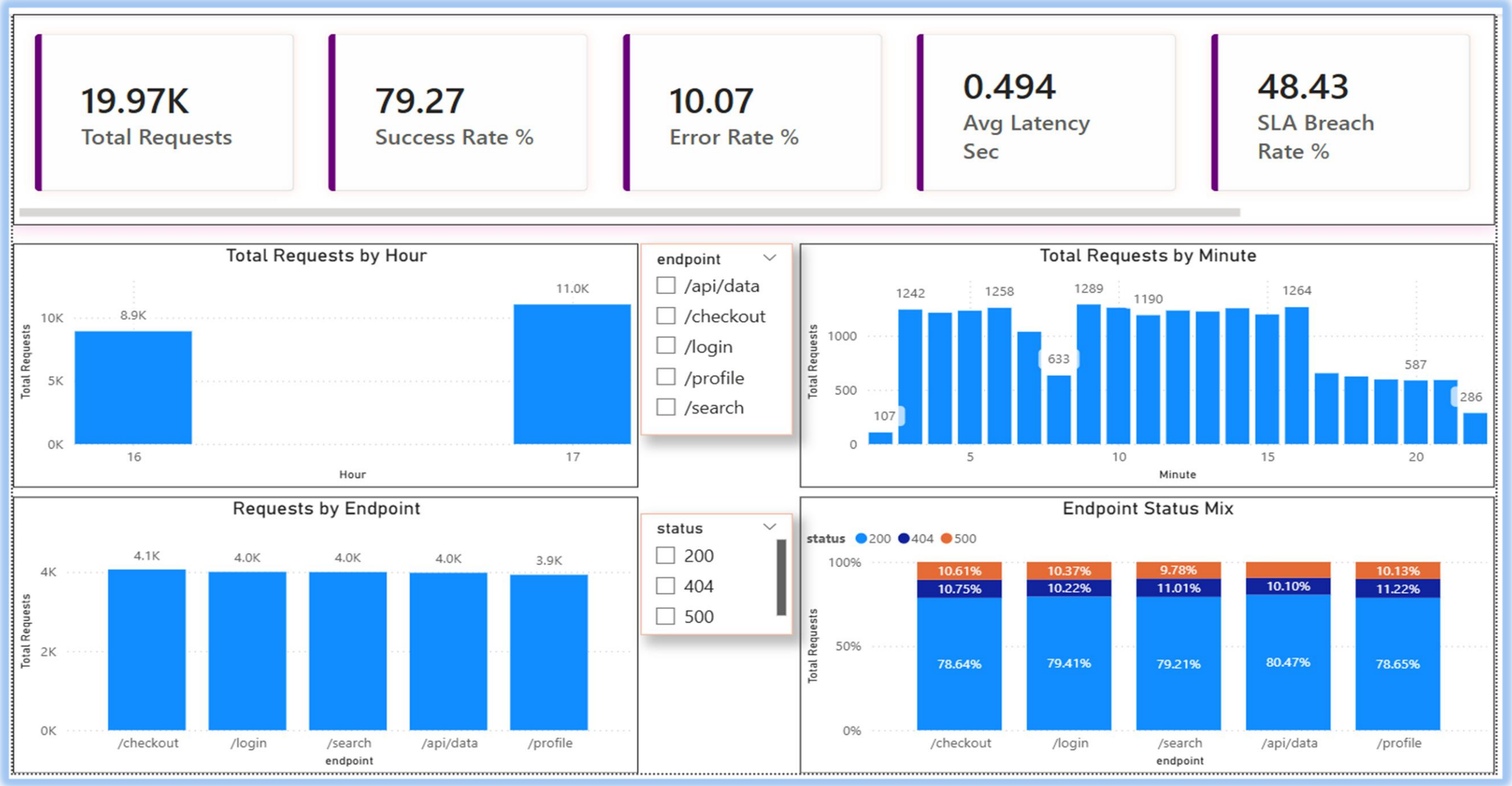
KPI + Diagnostics + Action Queries

KPI:

✓ ETL quality and freshness.

- Health score uses weighted reliability model.
- Master adds rolling trend and spike detection.
- Risk score prioritizes optimization work.

Power BI Dashboard



Observed KPI Results

- ✓ **Total Requests:** 19.97K
- ✓ **Success Rate:** 79.27%
- ✓ **Error Rate:** 10.07%
- ✓ **Average Latency:** 0.494 sec
- ✓ **SLA Breach Rate:** 48.43%

Meaning and Action

> Traffic has clear peak windows.

> Reliability is moderate, not ideal.

> SLA breach is the top concern.

> Fix high-risk endpoints first.

Key Outcomes

- ✓ Detects slow queries **BEFORE system failures.**
- ✓ Improves system reliability
- ✓ Helps identify root causes quickly
- ✓ Enables data-driven performance optimization
- ✓ Helps teams proactively monitor and optimize system performance
- ✓ Enables proactive monitoring and optimization

Current Strengths

I. Complete flow from generator to dashboard.
II. Clean and rejected paths are separated.
III. Rejected data is stored and can be analyzed.
IV. Risk scoring and root-cause actions add depth.

Next Improvements

I. Automate runs with scheduling and alerts.
II. Add partitioning for larger datasets.
III. Add security(RBAC) and query validation checks.
IV. Connect to real upstream sources.

Thank You

Instead of reacting to failures, this system helps detect issues early and act proactively.